POS rollout, rollback and support

This is a local implementation and verification run. It does not authorize deployment, enable new payment rails, change role assignments, approve country rules or perform live financial operations. The final verification matrix controls the release decision.

Before broader rollout

1. Review the run-specific changes against the saved dirty baseline, including the receipt page and scoped read/action contracts. Do not use HEAD alone to identify tested code.
2. Complete the unresolved final-source build and authorized browser checks in the final matrix. The 21 PostgreSQL cases have passed within their documented scope; French drawer rendering/filtering and bounded keyboard checks now pass. Cashier/manager journeys and broader responsive/accessibility checks still require representative accounts assigned through the existing role process.
3. Resolve the repository inventory gate and obtain authentic contract/program approvals, provider/country evidence, operational ownership and an approved pilot scope. Do not manufacture approval fixtures.
4. Provision and validate the production receipt secret through the normal secret process. Investigate historical bearer URLs in audit/backups, revoke affected active tokens where required, and preserve reviewed remediation provenance.
5. Exercise the exact offered product paths. Current cashier UI is cash with receipt NONE. Hardware, external delivery, electronic reversals, on-account UI, partial returns, cross-cashier transfer and offline enrollment/capture require separate complete acceptance evidence before they are offered.

No new schema migration was authored by these changes. Real integration uncovered the existing unapplied `20260909093000_add_suspense_posting_cycles` migration. The first approval covered only the named disposable database. After the user separately authorized the application change, the exact original SQL completed on local application database `stoquify_replay_next_20260830` at 04:42 UTC on 24 September. Application result verifies the column/default, validated check, replacement unique index, removal of the old index, matching completed Prisma history and preservation of unrelated schema/history. POS-R-10 is resolved for that target. Other environments still require their own reviewed migration rollout; no unrestricted pending-migration deployment occurred.

Keep `npm run build` out of ad-hoc verification because it deploys migrations. Use the reviewed `build:app` path with `BUILD_APP_SKIP_CLEAN=1`; Windows npm forwarding did not reliably preserve the CLI cleanup flag in this run. Inspect child/build packaging outcomes.

Cashier training

Situation	What to do	What it means
Charge is unavailable	Read the adjacent blocker. Add items, complete cash tender, select an active register/shift, or use Resume when requested.	A disabled button represents unmet prerequisites, not a reason to bypass them.
Another checkout owns the register	Finish/close the other checkout view and allow ownership to be reacquired normally.	Do not clear browser storage or force the lease. Closing a view is not closing a financial shift.

Situation	What to do	What it means
Prices or stock changed	Resume/review the cart, inspect the updated lines and total, then use the explicit next submit action.	“Cart reviewed” is not “sale recorded.” Reviewing cannot charge.
Previous sale outcome is unknown	Check the retained request. Keep the original request identifier and payload.	“No result found” does not prove the earlier request stopped. An explicit retry must use the same retained identity.
A recorded sale is recovered	Use its returned sale/order evidence and authorized receipt/report links.	Do not submit another sale merely because the original response was lost.
Connection is lost	Keep entered work and follow reconnect/review guidance.	A late response cannot silently restore expired authority.
Receipt is unavailable	Retry the receipt read or ask the store for a newly authorized link.	A receipt error does not undo or repeat the recorded sale.
Shift close is blocked	Resolve the stated retained/offline work and reconcile the count/variance through the existing process.	Do not manually edit financial rows to make the shift appear closed.

French equivalent guidance appears in the product's localized recovery states; future training material should preserve the same distinction between vérification, révision du panier and vente enregistrée.

An open session's absent final balance now displays the existing localized unavailable label rather than zero. This is a read-only presentation correction; the browser follow-up verifies the null and recorded-numeric states without closing a shift. A displayed unavailable value is not a recorded closing count.

Support investigation

Collect only the request reference, sale/order reference, approximate time, organization/location/terminal/session identifiers through authorized support channels, observed status and safe error code. Never paste receipt bearer URLs, credentials or customer contact details into tickets or logs.

Start with a read-only scoped outcome check. Compare the original request, cart revision, durable result, sale, payment, drawer/shift, stock and journal/event source references. NOT_FOUND alone is insufficient proof of rollback. An unresolved identity must not be silently replaced. A refund/void conflict requires investigation of the original sale and the current shift's unique opening drawer, balances and terminal ownership.

When evidence disagrees, retain the request and records, stop the affected workflow and escalate to the domain owner. Use an approved reconciliation or compensating operation; do not fabricate opening events, payment reversals, fiscal approval or successful registry states.

Rollback

The worktree was dirty before this run. Use the saved per-file baseline and run-specific diff/manifest to revert only a reviewed change; never reset the repository. Preserve receipts, original requests, audit evidence, posted money and stock history.

Do not restore the known unsafe behaviors: unsupported electronic reversals, unconditional wrong-drawer balance writes, unscoped drawer caches or audit bearer URLs. If a regression is discovered after deployment, disable the affected existing entry point through the normal controlled rollout mechanism and ship a corrective change. Do not delete financial data or reopen a closed shift as a rollback shortcut.

The completed posting-cycle migration is now part of the local application's recorded schema history. Any schema correction must preserve posting evidence and use a reviewed migration; do not blindly drop `postingCycle`, reinstate the old uniqueness rule or erase its Prisma history as a UI rollback.

Test artifacts and cleanup

`evidence/postgres-target.json` identifies the disposable database `stoquify_pos_audit_20260923_d1532fb1` on localhost:5432. Synthetic fixtures are retained for review. The bootstrap runner is provenance, not a command to rerun blindly; the existing-target runner validates the target before tests. The application database was not a financial test-write target; it was separately authorized for the exact schema/history migration documented above.

An operator may retire the disposable database only after explicitly confirming its exact identity and retention needs. This run does not drop it or any unrelated database. Development and attempted build output used separate directories, but the build wrapper nevertheless removed the previous generated `.next` output when npm dropped its cleanup flag; see the incident in the verification matrix. Source and isolated development output survived. Build-owned TypeScript changes and the run-owned development include were removed with baseline and equality checks, preserving all other configuration changes. Browser viewport overrides were reset after testing.